Version Control for Planetary Science: A Practical Guide to Git in Academic Software Development

William Byrne[1,2], Quentin Betton[2, 3], Christian Hüttig[1], Mario D'Amore[1]

[1]Institute of Space Research, German Aerospace Center (DLR), Berlin, Germany, William.Byrne@dlr.de; Christian.Huettig@dlr.de; Mario.DAmore@dlr.de
[2]Institute of Geological Sciences, Freie Universität Berlin, Berlin, Germany, byrnew02@zedat.fu-berlin.de
[3]Laboratoire de Planétologie et Géosciences, Nantes Université, Nantes, France, quentin.betton@etu.univ-nantes.fr

02.07.2026:View abstract on the EPSC2026 website

Plain Language Summary

Planetary science is becoming more reliant on programming, but many researchers still work in ways that make software easy to lose, hard to share, and stressful to discuss. This abstract explains how Git can help by introducing simple, realistic workflows that match different team sizes. For someone working alone, a small routine of saving and syncing changes can create a clear history and make mistakes easier to recover. For small teams (three to five people), using short-lived branches and pull requests adds lightweight peer review, improves communication, and reduces the risk of breaking software. For larger teams, adding a development branch and a basic testing suite supports steady collaboration while keeping a stable version available for users. Any software that aims to be used by more than one person, or used at significant temporal intervals, should adhere to minimal software engineering best practices, and version control is one of them [citation for this being best practice]. The goal is not to force one strict method, but to show that version control can be taught and adopted in academic settings. In planetary science, better Git habits can preserve knowledge, support teamwork, and build confidence in shared research software.

Abstract

Planetary scientists increasingly write their own software. Most do so alone, without version control, and (especially those early in their careers) with the quiet anxiety that comes from knowing their software may not survive advisor scrutiny. Git [1], software developed specifically to manage version control, has been the standard tool for collaborative software development in industry for decades, yet its adoption in academic research remains inconsistent [2, 3]. Not

because researchers lack the ability to use it, but because no one has made a concrete case for how it fits into the academic workflow. This work attempts to decrease the ill-perceived barrier to entry by providing three Git-based workflows of increasing complexity, designed for planetary science teams ranging from a single student to a large collaborative group.

For a single programmer, the entry point should be deliberately minimal. This will promote adoption and prevent confusion through unnecessary complexity. A researcher working alone commits to a single branch, using a handful of commands in a repeatable loop: `git fetch` to check for remote changes without overwriting local work, `git pull` to synchronize with the remote repository, `git add`, `git commit -m` with an appropriate commit message, and `git push` to update the remote repository with the current changes. The payoff is immediate: a complete, recoverable history of every change made to a codebase, along with a growing record of why those changes were made. In practice, this habit also improves software quality. Following a standardized comment format clearly communicates changes between versions.

For groups of two to five, we suggest a branching workflow that introduces the core of what makes Git genuinely powerful. Development happens on short-lived feature branches, each tied to a single tracked issue, while a protected `main` branch holds only tested, stable software. Merging back into the `main` branch happens through a pull request; a structured, documented conversation about whether the work is ready. This is where the workflow earns its place in an academic context. The pull request stage is peer review applied to software. Merge conflicts, which have a reputation for being intimidating, are in practice a straightforward task. Git has found two competing versions of the same line and wants a human to decide which one is correct. They become rarer the more regularly a team commits. Teams that adopt this pattern consistently report that it reduces the fear of breaking things, distributes ownership across contributors, and creates a trail of technical decisions that survives student turnover [4].

One can extend this logic to groups of five or more by inserting a `develop` branch between active development and the stable `main` branch. Feature branches merge into the `develop` branch continuously. The `develop` branch merges into the `main` branch only after passing a test suite. The result is a clear, standardized pattern of open development, followed by deliberate consolidation. This maps naturally onto how research projects are actually structured. Collaborators and users always have access to a working version of the software, and the test suite acts as a lightweight, automated form of the same quality control that pull requests provide at the individual feature level.

The goal of this abstract is not to prescribe a single correct approach to scientific software development. Version control philosophy is heavily debated (especially in industry [5]), and any team, academic or otherwise, will make team-specific adjustments that fit their culture and project structure. The narrower claim is that the academic programming culture, in planetary science and beyond, loses significant value to undocumented software, unrecoverable changes, and the low-grade embarrassment that leads researchers to preface every software-sharing interaction with an apology. Structured version control does not solve all of that, but it is a concrete, teachable, and immediately applicable step toward software that researchers feel comfortable standing behind [6, 7].

References

[1] Git Development Team, 2026, git-scm.com
[2] Blischak et al., 2016, PLoS Comput Biol
[3] Haaranen et al., 2015, ITiCSE '15
[4] Milliken et al., 2021, HICSS-54
[5] Driessen, 2010, nvie.com
[6] Ram, 2013, Source Code Biol Med
[7] Carver et al., 2022, PeerJ Comput Sci